

For this assignment, we worked with process synchronization and deadlock prevention using semaphores in Python. Specifically, we used Python's multiprocessing library to create multiple processes and synchronize how many of those processes were allowed to execute a shared section of code at the same time. The semaphore was implemented for the sake of controlling concurrent access to shared resources and preventing processes from interfering with one another while executing. To run the program, all that's needed is to run the main.py file.

The initialization value determines how many processes are allowed to access the protected section simultaneously. In this program, the semaphore was initialized with a value of 4, meaning only four processes were allowed to execute inside the critical section at one time. Once that limit was reached, any additional process attempting to enter the critical section was forced to wait until another process released the semaphore. For the sake of managing the semaphore automatically, the semaphore was implemented using:

```
# Semaphore controls access to this critical section
with semaphore:
```

Using this structure automatically acquires the semaphore once the process enters the protected section and automatically releases it once the process finishes execution. This helped simplify process synchronization while also helping avoid issues caused by forgetting to manually release the semaphore after execution.

To simulate actual process execution and make concurrent execution easier to observe, each process was assigned a random sleep interval using:

```
# Simulate process execution time  
sleep_time = random.randint(1, 3)
```

This allowed some processes to remain executing while others waited for the semaphore to become available. While the program was running, the output displayed which processes were waiting for the semaphore, which processes had acquired the semaphore, and which processes had released the semaphore after completing execution.

Deadlock prevention is important because deadlocks can cause processes to become permanently blocked while waiting for resources being used by other processes. When this happens, applications may freeze, system performance can decrease, and shared resources may become inaccessible. Improper synchronization can lead to deadlocks when multiple processes attempt to access shared resources without proper coordination, causing processes to wait indefinitely for one another. Several mechanisms can be used to prevent deadlocks, including semaphores, mutex locks, and resource ordering. In this assignment, semaphores were used to control how many processes could execute simultaneously and to force additional processes to wait until resources became available.

One of the biggest challenges encountered during implementation was understanding how semaphores manage synchronization between concurrent processes and how changing the semaphore initialization value affects concurrency, synchronization, and overall program behavior.